# DSA0309-NATURAL LANGUAGE PROCESSING

# ASSIGNMENT-2

**ASSIGNMENT 2 – CO2**

**CO2 Coverage:** N-grams Models of Syntax; Counting Words; Unsmoothed N-grams; Smoothing — Backoff, Deleted Interpolation; Entropy; English Word Classes; Tagsets for English; Part of Speech Tagging; Rule-Based Part of Speech Tagging; Stochastic Part of Speech Tagging; Transformation-Based Tagging

**Nature of Assignment: K DEEPTHI**

**REGISTER NO: 192472211**
**Duration:** 1 Week
**Total Marks:** 50
**Submission Date:** ____09 / 11 / 26_____

**Assignment Title: Build and Evaluate an N-gram Language Model and a Part-of-Speech Tagger for a Public Transport Enquiry Helpline (Total Marks – 50)**

Duration: 1 week

**Scenario — Speech Transcript Analytics for a Transport Enquiry Helpline**

A state transport corporation runs a telephone enquiry helpline that takes about four thousand calls a day about bus and train timings, routes, fares, cancellations and refunds. The calls are transcribed automatically, and the speech recogniser does not return one transcript but several competing hypotheses for every utterance: "two tickets to Vellore", "to tickets to Vellore" and "two tickets to Velore" all come back with similar acoustic scores. The recogniser cannot choose between them, because the decision is linguistic, not acoustic. Once a transcript is chosen it must be tagged before the origin, destination, date and fare can be pulled out of it, since "book", "return" and "stand" are each a noun in one call and a verb in the next.

You are the language-engineering team. You must build the n-gram language model that rescores the recogniser's hypotheses, measure it honestly with entropy and perplexity on held out data, design a tagset for the domain, and build and compare three part-of-speech taggers — rule-based, stochastic and transformation-based. Every claim you make about one model being better than another must rest on a measurement you made yourself.

**Planning data and constraints**

- Corpus: at least 20,000 transcribed utterances totalling at least 150,000 word tokens (real or synthetic, but documented), split 80 / 10 / 10 into training, development and test sets with no utterance appearing in more than one split.
- Recogniser output: at least 200 utterances for which five competing hypotheses and the correct

reference transcript are available, so that rescoring can be evaluated.

- Tagset (design and document your own): between 20 and 30 tags covering the open and closed word classes of the domain, with an explicit mapping to the Penn Treebank tagset.
- Annotated data: at least 3,000 tokens hand-tagged with your tagset, of which at least 500 tokens are tagged independently by two people so that agreement can be reported.
- Evaluation: a held-out set that is never used for training or for tuning any weight, and a fixed random seed for every experiment so that results can be reproduced.
- Any manual effort (transcription, tagset design, annotation, rule writing, testing) must be accounted for in your effort and cost discussion.

**Design Challenge (Day 4):**

The helpline is extended to metro and intercity rail services and begins to receive code-mixed speech, so the vocabulary grows sharply and more than fifteen per cent of the words in new calls are never seen in training. Re-evaluate your language model and your taggers and decide whether to retain, modify or redesign them.

# TASK 1 – Counting Words and Building the Vocabulary 3 Marks

**Guiding Enquiry**

What exactly is being counted when we count the words of a corpus, and how do those decisions change every model built on top of them?

**Investigate and interpret:**

- Distinguish type, token, lemma and word form, and report all four counts for your corpus; state how you treated case, punctuation, numerals, disfluencies and the utterance boundary markers <s> and </s>.
- Justify every tokenisation decision that changes the counts — whether "Vellore" and "vellore" are one type, whether "6:30" is one token or three — and show the effect of two different decisions on the vocabulary size.
- Plot the frequency of each word against its rank, state Zipf's law, and check how closely your corpus follows it; identify the words that deviate most and explain why the domain causes the deviation.
- Build the vocabulary with an unknown-word threshold, mapping rare words to <UNK>; report the vocabulary size at thresholds of 1, 2, 3 and 5 and the proportion of tokens each threshold sends to <UNK>.
- Measure the out-of-vocabulary rate of the development set against the training vocabulary and explain why this single number limits everything that follows.

**Evidence Expected**

Documented counting decisions, a Zipf plot for the corpus, and vocabulary and OOV figures at several thresholds.

# ANSWER

## 1.1 Meaning of type, token, lemma and word form

A token is one occurrence of a word in the corpus. If the sentence is “book two tickets”, it

contains three word tokens. A type is a distinct vocabulary item. In that sentence, “book”, “two” and “tickets” are three types. Repeated occurrences increase the token count but do not increase the type count.

A lemma is the dictionary base form that groups related inflected forms. For example, “book”, “books” and “booking” may be related to the lemma BOOK, depending on the lemmatisation policy. A word form is the exact surface form appearing in the transcript, such as “tickets” or “ticket”. Lemma counts are useful for linguistic analysis, while surface forms are important for recognising the exact words spoken.

| Term | Meaning | Transport example |
|---|---|---|
| Token | Every occurrence in text | “bus bus timing” has 3 tokens |
| Type | Unique surface item | “bus bus timing” has 2 types |
| Lemma | Base dictionary form | “tickets” → TICKET |
| Word form | Observed written form | “tickets”, “ticket” |

## 1.2 Tokenisation decisions

Tokenisation determines where one word ends and another begins. The report must state whether case is normalised, how punctuation is handled, whether numerals are retained, how speech disfluencies are represented and whether sentence-boundary markers are counted.

Case: one possible policy is to lowercase all words so “Vellore” and “vellore” map to one type. Keep a copy of the original transcript if capitalisation is needed for named-entity analysis.

Punctuation: remove punctuation that does not carry meaning, but document how question marks or abbreviations are handled.

Numerals and times: decide whether “6:30” is one token or is split into “6”, “:” and “30”. Apply the same rule to all splits.

Disfluencies: either retain fillers such as “uh” and repetitions or remove them consistently. For speech analytics, retaining them may help represent natural calls.

Boundaries: use <s> and </s> to mark utterance starts and ends. State whether they are included in token totals and n-gram counts.

To show the effect of tokenisation, process the same corpus twice. Policy A may lowercase words and keep a time such as “6:30” as one token. Policy B may preserve case and split the time into three tokens. Report the resulting vocabulary sizes as measured values; do not assume the two policies produce a particular difference.

| Counting item | Policy A | Policy B |
|---|---|---|
| Case | Lowercase | Preserve case |
| Time 6:30 | One token | Three tokens: 6, : , 30 |
| Vellore / vellore | One type | Two types |
| Vocabulary size | [measured V_A] | [measured V_B] |

## 1.3 Zipf’s law

Zipf’s law states approximately that the frequency of a word is inversely proportional to its rank in a frequency-ordered list. If the most frequent word has rank 1, a word at rank r is expected to have a frequency roughly proportional to 1/r. In a log-log plot, this often appears approximately as a straight descending line.

Create a frequency table by counting each word, sort words from highest to lowest frequency, assign ranks and plot rank against frequency. A log-log plot is useful because word frequencies cover a wide range. In a transport corpus, words such as “bus”, “train”, “ticket”, “to” and “timing” may occur frequently. Station names and rare route names may deviate because the domain contains many specific proper nouns.

## 1.4 Unknown-word threshold and OOV rate

A rare-word threshold maps words occurring fewer than a selected number of times in training to the special token <UNK>. For threshold 1, only words absent from training are unknown; for threshold 2, words seen once are also mapped to <UNK>. Repeat this process for thresholds 1, 2, 3 and 5.

| Threshold | Vocabulary size | Training tokens mapped to <UNK> | Development OOV rate |
|---|---|---|---|
| 1 | [value] | [value] | [value] |
| 2 | [value] | [value] | [value] |
| 3 | [value] | [value] | [value] |
| 5 | [value] | [value] | [value] |

OOV rate is the percentage of development tokens that are not represented in the training vocabulary. It is calculated as: OOV rate = (number of development tokens mapped to <UNK> / total development tokens) × 100. A high OOV rate limits language-model prediction because the model cannot distinguish unseen names unless it has a suitable unknown-word strategy.

## 1.5 Task 1 conclusion

The final report for this task should include all four counts, the written tokenisation policy, two vocabulary-size comparisons, a Zipf plot, threshold tables and the development OOV rate. These decisions must remain fixed when comparing later models.

# TASK 2 – Unsmoothed N-gram Models 4 Marks

**Guiding Enquiry**
How is the probability of a word sequence estimated from counts alone, and where does that estimate break down?

**Investigate and interpret:**

- State the chain rule for the probability of a word sequence and derive the Markov assumption that turns it into a bigram and then a trigram model, saying precisely what is being assumed away at each step.
- Give the maximum likelihood estimator for unigram, bigram and trigram probabilities, and build all three models from your training split.
- Compute by hand the unsmoothed probability of at least three helpline utterances, showing every factor; choose one utterance that is perfectly reasonable English but receives probability zero,

and identify the exact bigram responsible.

- Report the sparsity of your models: the proportion of possible bigrams and trigrams actually observed, and the proportion of development-set bigrams and trigrams never seen in training.
- Use the models as generators: sample ten utterances from the unigram, bigram and trigram models and comment on what the increase in order buys and what it costs.

**Evidence Expected**

Working unsmoothed models, hand-computed sentence probabilities including a zero, and quantified sparsity figures.

# ANSWER

## 2.1 Chain rule and Markov assumption

The chain rule expresses the probability of a sequence as a product of conditional probabilities:

$P(w1, w2, \ldots, wn) = P(w1) \times P(w2|w1) \times P(w3|w1,w2) \times \ldots \times P(wn|w1,\ldots,w(n-1))$.

This exact expression uses the complete preceding history. An n-gram model simplifies it by assuming that only a limited number of previous words are needed to predict the next word. A unigram model ignores previous words. A bigram model uses only the immediately previous word. A trigram model uses the previous two words. The simplification makes estimation practical, but it loses longer-range grammatical and semantic information.

## 2.2 Maximum likelihood estimates

The maximum likelihood estimate (MLE) uses relative frequency. Let C(w) be the count of a word, C(w1,w2) the count of a bigram and C(w1,w2,w3) the count of a trigram.

Unigram: $P(w) = C(w) / N$

Bigram: $P(w2 \mid w1) = C(w1,w2) / C(w1)$

Trigram: $P(w3 \mid w1,w2) = C(w1,w2,w3) / C(w1,w2)$

N is the total number of word tokens under the chosen boundary convention. The unigram model estimates the general frequency of words. The bigram model estimates the next word from one preceding word. The trigram model uses a two-word context.

## 2.3 Worked illustrative probability

Consider the illustrative utterance “book two tickets”. Under a bigram model, its probability with a start marker can be written as:

P(book | <s>) × P(two | book) × P(tickets | two) × P(</s> | tickets)

For a real corpus, replace each factor with its training count divided by the corresponding context count. For example, P(two | book) = C(book, two) / C(book). The report must show the actual count and denominator for every factor.

An unsmoothed model assigns probability zero if any required n-gram has count zero. A sentence may be perfectly understandable but still receive zero probability because one combination was not observed in training. Identify the exact unseen bigram or trigram from your own training data and show its zero count.

## 2.4 Sparsity

The number of possible bigrams grows approximately with the square of the vocabulary size, while possible trigrams grow approximately with its cube. Only a small fraction of these combinations appear in ordinary text. Sparsity can be reported as observed n-gram types divided by possible n-gram types. For a development set, count the n-grams that do not occur in training and divide by the total development n-gram occurrences.

| **Measure** | **Bigram** | **Trigram** |
|---|---|---|
| Possible combinations | [value] | [value] |
| Observed training types | [value] | [value] |
| Observed proportion | [value] | [value] |
| Unseen development n-grams | [value] | [value] |
| Development unseen proportion | [value] | [value] |

## 2.5 Generating sample utterances

Generate ten sample sequences from each model using a fixed random seed. A unigram generator chooses words independently, so its output usually lacks word order. A bigram generator tends to produce locally plausible pairs. A trigram generator can represent somewhat longer local patterns, but it may become trapped when a context has no suitable continuation. Stop generation at </s> or a documented maximum length.

Task 2 conclusion: compare the three models in terms of context, fluency, sparsity and zero-probability failures. Include hand calculations and sample generated utterances.

## TASK 3 – Smoothing and Discounting 5 Marks

**Guiding Enquiry**

How can probability mass be moved from what was seen to what was not, without destroying what the counts tell us?

**Investigate and interpret:**

- Explain why a zero probability is a modelling failure rather than a fact about the language, and state the condition any smoothing method must satisfy for the distribution to remain valid.
- Implement add-one (Laplace) and add-k smoothing; for a bigram with a left context of your choice, tabulate the original counts, the effective (discounted) counts and the resulting probabilities, and show exactly how much mass was taken from the seen bigrams.
- Explain why add-one smoothing discounts so severely when the vocabulary is large, with the arithmetic for one context from your own corpus.
- Implement Good–Turing discounting: build the frequency-of-frequencies table $N_r$, compute the revised counts c*, estimate the probability of all unseen bigrams, and explain how the high-frequency tail is handled.
- Compare the methods on the development set by perplexity and by how reasonable the estimate for a specific unseen bigram looks, and state which you would deploy and why.

**Evidence Expected**

Correctly implemented smoothing methods with discount tables, a Good–Turing $N_r$ table, and a measured comparison.

## ANSWER

## 3.1 Why smoothing is required

A zero count does not prove that a word sequence is impossible in the language. It often means only that the training corpus was too small to contain that sequence. Smoothing assigns some probability mass to unseen events while reducing the probabilities of events that were observed. A valid conditional probability distribution must contain non-negative probabilities and the probabilities of all possible next words for a fixed context must sum to one.

## 3.2 Add-one (Laplace) smoothing

Laplace smoothing adds one to every possible continuation count. If the vocabulary size is V,

the smoothed bigram estimate is:

P_Laplace(w | h) = (C(h,w) + 1) / (C(h) + V)

This prevents zero probabilities. However, when V is large, adding one to every possible word adds a large total amount of artificial count. This can take too much probability mass away from frequent, observed continuations.

## 3.3 Add-k smoothing

Add-k smoothing adds a smaller constant k to each possible continuation:

P_add-k(w | h) = (C(h,w) + k) / (C(h) + kV)

When k is between zero and one, the discount is less severe than add-one. The value of k should be selected using development data, not the test set. Report the selected k and the development procedure used to choose it.

## 3.4 Worked illustrative calculation

Suppose a context has count 10 and the vocabulary contains 100 words. Under add-one smoothing, the denominator becomes 10 + 100 = 110. If one continuation occurred 4 times, its smoothed probability is 5/110. If another occurred zero times, its probability is 1/110. These figures are only an illustration; use a context and counts from your own corpus in the final experiment.

| **Continuation** | **Original count** | **Add-one count** | **Add-one probability** |
|---|---|---|---|
| tickets | [C1] | [C1+1] | [calculated] |
| tomorrow | [C2] | [C2+1] | [calculated] |
| unseen word | 0 | 1 | [calculated] |

## 3.5 Good–Turing discounting

Good–Turing discounting uses the frequency of frequencies. N_r is the number of distinct n-gram types that occur exactly r times. For an n-gram with count r, the adjusted count is commonly estimated by:

c* = (r + 1) × N_(r+1) / N_r

The estimated total probability mass for unseen events is related to N1/N, where N1 is the number of n-gram types seen once and N is the total number of n-gram occurrences. The method redistributes some probability from observed events to unseen events. In practice, the high-frequency tail may require smoothing of the N_r estimates or a switch back to raw counts when the frequency-of-frequencies becomes unreliable.

| **r** | **N_r from corpus** | **N_(r+1)** | **Adjusted count c*** |
|---|---|---|---|
| 0 | [unseen types] | [N1] | [unseen mass estimate] |
| 1 | [N1] | [N2] | 2N2/N1 |
| 2 | [N2] | [N3] | 3N3/N2 |
| 3 | [N3] | [N4] | 4N4/N3 |
| 4+ | [values] | [values] | [smoothed / raw-tail policy] |

## 3.6 Comparing smoothing methods

Evaluate add-one, add-k and Good–Turing on the same development set with the same vocabulary and <UNK> policy. Compare perplexity and inspect the probability assigned to a meaningful unseen bigram. The preferred method should give a lower development perplexity without assigning implausibly large probability to rare events. Report the exact measured values and explain the choice.

| **Method** | **Development perplexity** | **Unseen-bigram probability** | **Decision** |
|---|---|---|---|
| Add-one | [value] | [value] | [comment] |
| Add-k | [value] | [value] | [comment] |
| Good–Turing | [value] | [value] | [comment] |

# TASK 4 – Backoff and Deleted Interpolation 4 Marks

**Guiding Enquiry**

When the trigram has never been seen, how should the model fall back on the shorter contexts it does know?

**Investigate and interpret:**

- Define backoff and interpolation and state the essential difference: backoff consults a shorter context only when the longer one fails, interpolation always combines all orders.
- Implement Katz backoff with Good–Turing discounting, explaining the role of the normalising weight α and why the model is not a valid distribution without it; also implement stupid backoff and say why its output is a score and not a probability.
- Implement simple linear interpolation, then deleted interpolation: hold out part of the training data, estimate the λ weights on it, and report the weights you obtain for at least three context-count bands.
- Explain why the λ weights should depend on the count of the context, and show with one worked example from your corpus how a low-count context shifts weight onto the lower orders.
- Compare backoff and interpolation on the held-out set by perplexity, and hand-trace one utterance through both so that the difference in treatment of a single unseen trigram is visible.

**Evidence Expected**

Working backoff and interpolation models, estimated λ weights with the held-out procedure described, and a traced comparison.

# ANSWER

## 4.1 Backoff and interpolation

Backoff uses a longer context when it has reliable evidence. If a trigram is missing, the model falls back to a bigram; if the bigram is also unavailable, it may use a unigram. Interpolation always combines probability estimates from multiple orders, even when the highest-order n-gram exists.

The main difference is therefore conditional fallback versus weighted combination. Backoff can preserve strong higher-order evidence, while interpolation makes use of lower-order evidence at every prediction.

## 4.2 Katz backoff

Katz backoff discounts observed n-grams and uses a lower-order model for unseen events. The backoff weight α is a normalising factor. It ensures that the probability mass assigned to unseen

continuations is distributed correctly after the discounted seen events have been accounted for. Without suitable normalisation, the conditional probabilities may not sum to one.

For a trigram, use the trigram probability if the trigram has sufficient count. Otherwise, back off to the bigram estimate, multiplied by the relevant α. If the bigram is also unavailable, back off again to a unigram. Explain the discounting and normalisation steps used in your implementation.

## 4.3 Stupid backoff

Stupid backoff uses a simple fixed multiplier when it moves to a lower-order n-gram. It is computationally simple and can rank candidate sequences. However, its scores are not guaranteed to sum to one, so they are scores rather than true probabilities. Perplexity should not be interpreted as if these unnormalised scores were a valid probability distribution.

## 4.4 Linear interpolation

Linear interpolation combines unigram, bigram and trigram estimates:

$$P(w_i \mid w_{(i-2)}, w_{(i-1)}) =$$

$$\lambda 3\ P(w_i \mid w_{(i-2)}, w_{(i-1)}) +$$

$$\lambda 2\ P(w_i \mid w_{(i-1)}) + \lambda 1\ P(w_i)$$

The weights must be non-negative and satisfy $\lambda 1 + \lambda 2 + \lambda 3 = 1$. A high $\lambda 3$ gives more importance to trigram evidence; $\lambda 1$ gives more importance to unigram evidence.

## 4.5 Deleted interpolation

Deleted interpolation estimates weights by temporarily holding out part of the training data. For each held-out n-gram occurrence, compare the unigram, bigram and trigram estimates calculated without that occurrence. Give the occurrence's weight to the order that provides the strongest estimate. Aggregate these decisions, optionally by context-count bands, and normalise the totals to obtain λ values.

| **Context-count band** | **λ1 unigram** | **λ2 bigram** | **λ3 trigram** | **Reason** |
|---|---|---|---|---|
| Low count | [value] | [value] | [value] | Less reliable long context |

| | | | | |
|---|---|---|---|---|
| Medium count | [value] | [value] | [value] | Mixed evidence |
| High count | [value] | [value] | [value] | More reliable trigram evidence |

Low-count contexts generally need more support from lower-order models because the longer context has little evidence. Use one actual context from the corpus to show its counts and resulting weighted probability.

## 4.6 Evaluation and hand trace

Compare Katz backoff and deleted interpolation on the same held-out data. For one utterance, show the probability path for a seen trigram and for an unseen trigram. In the unseen case, show exactly which shorter context is consulted and how the backoff weight is applied. For interpolation, show all weighted components.

| **Model** | **Held-out perplexity** | **Memory** | **Speed** | **Notes** |
|---|---|---|---|---|
| Katz backoff | [value] | [value] | [value] | [comment] |
| Linear / deleted interpolation | [value] | [value] | [value] | [comment] |

# TASK 5 – Entropy, Cross-Entropy and Perplexity 4 Marks

**Guiding Enquiry**

How is a language model measured, and what does the number actually mean?

**Investigate and interpret:**

- Define entropy, cross-entropy and perplexity, state the relation between them, and explain in plain words what a perplexity of 90 tells an engineer about the model.
- Compute the entropy of at least three distributions from your own model by hand — including one that is nearly uniform and one that is sharply peaked — and explain what the difference means for prediction.
- Compute the cross-entropy and perplexity of the held-out set under each model built in Tasks 2 to 4, and present the results in one comparable table; state the per-word and per-utterance figures separately.
- Explain why perplexity may only be compared between models that share a vocabulary and the same treatment of <UNK>, and demonstrate the distortion with a deliberately unfair comparison of your own.
- Use your 200 recogniser hypothesis sets: rescore each set with your best model, report the word error rate before and after rescoring, and state whether perplexity improvements translated into error reductions.

**Evidence Expected**

Correct entropy calculations, a comparable perplexity table, and a rescoring experiment relating perplexity to word error rate.

# ANSWER

## 5.1 Definitions

Entropy measures the uncertainty of a probability distribution. For outcomes with probabilities p(x), entropy is:

$H(X) = -\Sigma_x p(x) \log_2 p(x)$

A sharply peaked distribution has low entropy because most probability is concentrated on a small number of outcomes. A nearly uniform distribution has higher entropy because many outcomes are similarly likely.

Cross-entropy measures how well a model distribution predicts data drawn from a true distribution. For a sequence of N words, the average cross-entropy in bits per word is:

$H_{cross} = -(1/N) \Sigma_i \log_2 P_{model}(w_i \mid history_i)$

Perplexity is the exponentiation of cross-entropy. With base 2, $PP = 2^{(H_{cross})}$. Lower perplexity generally means the model is less uncertain about the next word on the evaluation data. A perplexity of 90 can be interpreted as an effective branching factor of about 90 equally

likely choices per prediction; it does not mean the model literally stores only 90 words.

## 5.2 Hand calculation of entropy

For a two-outcome distribution with probabilities 0.5 and 0.5, entropy is 1 bit. For a sharply peaked distribution such as 0.9 and 0.1, entropy is lower because one outcome is much more predictable. Use three distributions derived from your own model: one nearly uniform, one sharply peaked and one intermediate. Show every −p log2 p term.

| **Distribution** | **Probabilities** | **Entropy (bits)** | **Interpretation** |
|---|---|---|---|
| Nearly uniform | [p1, p2, …] | [calculated] | Higher uncertainty |
| Sharply peaked | [p1, p2, …] | [calculated] | Lower uncertainty |
| Intermediate | [p1, p2, …] | [calculated] | Moderate uncertainty |

## 5.3 Comparable evaluation

Evaluate every model on exactly the same held-out utterances, vocabulary, tokenisation and <UNK> policy. Report per-word cross-entropy and perplexity. Also report per-utterance averages if required, clearly explaining how utterance boundaries are treated. A table makes comparisons easier.

| **Model** | **Cross-entropy / word** | **Perplexity / word** | **Cross-entropy / utterance** | **Perplexity / utterance** |
|---|---|---|---|---|
| Unigram | [value] | [value] | [value] | [value] |
| Bigram unsmoothed / smoothed | [value] | [value] | [value] | [value] |
| Trigram | [value] | [value] | [value] | [value] |
| Katz backoff | [value] | [value] | [value] | [value] |
| Deleted interpolation | [value] | [value] | [value] | [value] |

## 5.4 Why vocabulary must match

Perplexity comparisons are fair only when models use the same vocabulary and the same unknown-word mapping. If one model maps many rare words to <UNK> while another retains them as separate words, their prediction problems are different. An unfair comparison can

make one model appear better or worse simply because its vocabulary is easier or harder. Demonstrate this by evaluating one model with two different vocabulary policies. Label the comparison as deliberately unfair and explain why it cannot support a model-selection claim.

## 5.5 Rescoring recogniser hypotheses

For each of at least 200 utterances, the recogniser supplies five candidate transcripts and a reference transcript. Score each candidate with the selected language model and rank them. Word error rate (WER) is calculated using substitutions (S), deletions (D), insertions (I) and the number of reference words (N):

$$WER = (S + D + I) / N \times 100$$

Measure WER before rescoring using the recogniser's original top hypothesis and after rescoring using the language-model-selected candidate. Perplexity improvement does not guarantee lower WER: a model may assign high probability to fluent wording that changes an important station name or number. Report both measures.

| **Evaluation** | **Before rescoring** | **After rescoring** |
|---|---|---|
| Word error rate | [value]% | [value]% |
| Correctly selected reference / candidates | [value] | [value] |

# TASK 6 – English Word Classes and Tagset Design 4 Marks

**Guiding Enquiry**

What are the word classes of English, and how many distinctions should a tagset for this helpline actually make?

**Investigate and interpret:**

- Distinguish open and closed word classes, list the members of the closed classes that occur in your corpus, and explain why the closed classes matter disproportionately for tagging.
- Define the major open classes with the morphological and distributional tests used to identify them, and apply those tests to at least five ambiguous words from your corpus.
- Design your tagset of 20 to 30 tags, justify each distinction you chose to make and at least three you chose not to make, and give the full mapping to the Penn Treebank tagset.
- Annotate your 3,000-token sample, report the tag frequency distribution, and give the inter-annotator agreement on the doubly annotated portion as a raw percentage and as Cohen's kappa.
- Measure the ambiguity your tagset leaves: the proportion of word types and of word tokens with more than one possible tag, and the average number of tags per ambiguous token.

**Evidence Expected**

A justified tagset with a Penn Treebank mapping, an annotated sample with agreement figures, and a measurement of residual ambiguity.

# ANSWER

## 6.1 Open and closed word classes

Open classes can accept new members. The major open classes are nouns, verbs, adjectives and adverbs. Transport examples include "ticket", "station", "cancel", "depart", "available" and "quickly". Proper nouns such as "Vellore" and "Chennai" are especially important in this domain.

Closed classes contain a relatively fixed set of words. They include determiners, pronouns, prepositions, conjunctions, auxiliaries and particles. Examples include "the", "a", "to", "from", "is", "will", "and" and "it". Closed-class words occur frequently and provide grammatical structure, so confusing them can affect the interpretation of a sentence.

## 6.2 Morphological and distributional tests

A noun may accept plural marking or occur after a determiner. A verb may show tense or agreement forms and occur after a subject. An adjective may modify a noun or follow a linking verb. An adverb may modify a verb, adjective or clause. These tests are clues rather than

absolute rules; the word's context must also be considered.

| Ambiguous word | Possible class | Example context | Evidence to inspect |
|---|---|---|---|
| book | Noun / verb | "book a ticket" / "the book" | Position and determiner/verb pattern |
| return | Noun / verb / adjective | "return fare" / "return tomorrow" | Nearby words and syntactic role |
| stand | Noun / verb | "bus stand" / "please stand" | Determiner or verb context |
| fast | Adjective / adverb | "fast train" / "travel fast" | Word modified |
| next | Adjective / adverb-like determiner | "next bus" / "what next?" | Position and function |

## 6.3 Designing a domain tagset

The assignment requires 20–30 tags and an explicit mapping to the Penn Treebank (PTB) tagset. A tagset should distinguish grammatical categories that affect downstream interpretation, but it should not create unnecessary distinctions that annotators cannot apply consistently.

The following is a proposed 24-tag starting point. Adapt it to the words actually present in the corpus and document every change.

| Domain tag | Meaning | Approximate PTB mapping |
|---|---|---|
| NOUN | Common noun | NN / NNS |
| PROPN | Proper noun, place or service name | NNP / NNPS |
| VERB | Base or non-third-person verb | VB |
| VERB3 | Third-person singular verb | VBZ |

| | | |
|---|---|---|
| VERBP | Past-tense verb | VBD |
| VERBG | Gerund / present participle | VBG |
| VERBN | Past participle | VBN |
| ADJ | Adjective | JJ / JJR / JJS |
| ADV | Adverb | RB / RBR / RBS |
| DET | Determiner | DT / PDT / WDT |
| PRON | Pronoun | PRP / PRP$ / WP |
| PREP | Preposition | IN |
| CONJ | Coordinating conjunction | CC |
| AUX | Auxiliary / modal | MD / auxiliary VB forms |
| PART | Particle | RP / TO where defined |
| NUM | Number / numeral | CD |
| INTJ | Interjection / filler | UH |
| PUNCT | Punctuation | punctuation tags |
| NEG | Negation marker | RB / DT, depending on form |
| WH | Wh-word | WDT / WP / WRB |
| DATE | Date expression | CD / domain-specific mapping |
| TIME | Time expression | CD / domain-specific mapping |
| FARE | Fare amount or fare category | CD / NN, domain-specific |
| UNK | Unresolved or unknown token | No direct PTB equivalent |

The mapping is many-to-one in places because a compact domain tag may combine several PTB distinctions. Explain why the mapping is appropriate. At least three distinctions not made should also be justified; for example, separate comparative and superlative adjective tags may

be unnecessary if the downstream task does not use them.

## 6.4 Annotation and agreement

Hand-tag at least 3,000 tokens. At least 500 tokens must be independently tagged by two annotators. Calculate raw agreement as the number of matching tags divided by the total double-annotated tokens. Cohen's kappa corrects for agreement expected by chance:

$\kappa = (P_o - P_e) / (1 - P_e)$

P_o is observed agreement and P_e is expected chance agreement based on the annotators' tag distributions. Report both raw percentage and κ. Also report tag frequencies and ambiguity: the percentage of word types and tokens that can receive more than one tag, plus the average number of candidate tags for ambiguous tokens.

| Annotation measure | Result |
|---|---|
| Total hand-tagged tokens | [at least 3,000] |
| Double-annotated tokens | [at least 500] |
| Raw agreement | [value]% |
| Cohen's kappa | [value] |
| Ambiguous word types | [value]% |
| Ambiguous word tokens | [value]% |
| Average tags per ambiguous token | [value] |

**TASK 7 – Rule-Based Part-of-Speech Tagging 4 Marks**

**Guiding Enquiry**

How far can hand-written rules take a tagger, and what makes writing them hard?

**Investigate and interpret:**

- Describe the two-stage architecture of a rule-based tagger: a dictionary that assigns every possible tag, then constraints that remove the impossible ones.
- Build the tag dictionary from your annotated data and report how many words receive one, two, three or more candidate tags.
- Write at least fifteen disambiguation rules in an explicit condition-action notation, each stating the context it inspects; include rules that use the preceding tag, the following word and the word's own suffix.
- Evaluate the tagger on the held-out set, report overall accuracy and accuracy on ambiguous tokens only, and explain why the second number is the one that matters.
- Analyse the failures: give the five commonest error types, show one case where two of your rules conflict, and explain how rule ordering resolved it and what that costs in maintainability.

**Evidence Expected**
A documented rule set with a tag dictionary, measured accuracy on ambiguous tokens, and a rule-conflict analysis.

# ANSWER

## 7.1 Two-stage architecture

A rule-based POS tagger commonly uses two stages. First, a dictionary assigns every possible tag to each known word. For example, "book" may receive both NOUN and VERB. Second, contextual constraints remove tags that are unlikely or impossible in the current sentence.

The dictionary is built from the annotated corpus. For each word, count the tags observed with it. Report how many words have one candidate tag, two candidate tags, three candidate tags or more than three.

## 7.2 Example condition-action rules

Rules should be written in a clear condition-action form. The following are illustrative rule patterns; verify and adapt them using your tagset and corpus.

IF the previous tag is PRON and the current candidate is VERB, KEEP VERB.

IF the previous tag is DET and the current candidate is NOUN, KEEP NOUN.

IF the previous tag is PREP and the current candidate is NOUN or PROPN, KEEP the nominal

tag.

IF the next word is a determiner and the current candidate is VERB, prefer NOUN when supported by the dictionary.

IF the word ends in “-ing” and follows an auxiliary, prefer VERBG.

IF the word ends in “-ed” and follows an auxiliary, prefer VERBN or VERBP according to context.

IF the word ends in “-ly”, consider ADV, unless the dictionary provides stronger evidence.

IF the word is a known station name, assign PROPN.

IF the word is a numeral followed by a time marker, consider TIME or NUM.

IF the previous tag is AUX, prefer a verb form.

IF the current word is “to” before a base verb, assign PART/TO according to the chosen tagset.

IF the current word is “from” or “to” before a place name, assign PREP.

IF the current word is “and” or “or”, assign CONJ.

IF the current word is a recognised filler, assign INTJ.

IF the word is unseen and capitalised, consider PROPN, but do not rely on capitalisation alone.

The assignment asks for at least fifteen rules. Include the preceding tag, following word and suffix evidence, and state the exact condition and action for each rule.

## 7.3 Evaluation and rule conflicts

Evaluate on held-out data. Overall accuracy is the number of correctly tagged tokens divided by all evaluated tokens. Accuracy on ambiguous tokens is especially informative because unambiguous dictionary entries are comparatively easy. Report the five most common error types with examples.

Two rules may conflict. For example, a suffix rule may suggest ADV while a context rule suggests ADJ. Specify rule priority and show how the ordering resolves the conflict. Rule ordering can improve results, but a long ordered list becomes harder to maintain when the vocabulary or domain changes.

| **Measure** | **Result** |
|---|---|
| Dictionary entries | [value] |
| Words with 1 candidate tag | [value] |
| Words with 2 candidate tags | [value] |
| Words with 3+ candidate tags | [value] |
| Overall held-out accuracy | [value]% |

| | |
|---|---|
| Ambiguous-token accuracy | [value]% |
| Five common error types | [insert list] |

## TASK 8 – Stochastic (HMM) Tagging and the Viterbi Algorithm 5 Marks Guiding Enquiry

How does a probabilistic tagger choose a whole tag sequence rather than one tag at a time?

**Investigate and interpret:**

- Derive the HMM tagging equation from Bayes' rule, state the two independence assumptions it rests on, and identify the transition and emission terms.
- Estimate the transition and emission probabilities from your annotated data, state how you smoothed them, and show that each transition row sums to one.
- Hand-trace the Viterbi algorithm on one ambiguous helpline utterance of at least five words: fill every trellis cell, keep a back-pointer for each, recover the best tag sequence and give its probability.
- Compare Viterbi against the most-frequent-tag baseline and against scoring every tag sequence exhaustively; report the accuracy of each and the number of operations the three require for your sentence.
- Explain how unknown words are tagged by an HMM — suffix evidence, capitalisation, an open-class prior — implement one such method, and report its accuracy on the out of-vocabulary tokens alone.

**Evidence Expected**

A correctly estimated HMM, a fully traced Viterbi trellis with back-pointers, and a measured treatment of unknown words.

## ANSWER

## 8.1 Hidden Markov Model

An HMM treats the POS tags as hidden states and the observed words as emissions. The goal is to find the most probable tag sequence for a given word sequence. Using Bayes' rule and simplifying assumptions, the best sequence is:

$$T^* = \text{argmax}_T \, P(T \mid W)$$

$$\approx \text{argmax}_T \, \Pi_i \, P(t_i \mid t_{(i-1)}) \, P(w_i \mid t_i)$$

The transition probability $P(t_i \mid t_{(i-1)})$ describes how likely one tag follows another. The emission probability $P(w_i \mid t_i)$ describes how likely a word is under a tag. The common assumptions are that the current tag depends only on the previous tag, and the current word

depends only on its current tag.

## 8.2 Estimating probabilities

Estimate transition probabilities from adjacent tag counts and emission probabilities from word-tag counts:

$P(t_i \mid t_{(i-1)}) = C(t_{(i-1)}, t_i) / C(t_{(i-1)})$

$P(w_i \mid t_i) = C(t_i, w_i) / C(t_i)$

Apply smoothing so unseen transitions and emissions do not force the entire sequence probability to zero. For every transition row, verify that probabilities over all possible next tags sum to approximately one. State the smoothing method and any unknown-word treatment.

## 8.3 Viterbi algorithm

Viterbi finds the most probable sequence efficiently using dynamic programming. It stores the best score for each possible tag at each word position and a back-pointer to the preceding tag that produced that score.

$V[t, j] = \max_i \{ V[t-1, i] \times P(tag_j \mid tag_i) \times P(word_t \mid tag_j) \}$

In a practical implementation, use logarithms to avoid numerical underflow. Multiplication becomes addition and the maximum-probability path can be recovered by following back-pointers from the best final tag.

## 8.4 Required hand trace

Choose an ambiguous utterance of at least five words from the helpline corpus. Create a trellis table with one column per word and one row per possible tag. Fill each cell with its best probability or log-score and record the back-pointer. At the end, select the best final tag and trace backward to recover the complete sequence.

| **Position / word** | **Candidate tag 1 score** | **Candidate tag 2 score** | **Candidate tag 3 score** | **Back-pointer** |
|---|---|---|---|---|
| 1: [word] | [score] | [score] | [score] | [tag] |
| 2: [word] | [score] | [score] | [score] | [tag] |
| 3: [word] | [score] | [score] | [score] | [tag] |
| 4: [word] | [score] | [score] | [score] | [tag] |

| | | | | |
|---|---|---|---|---|
| 5: [word] | [score] | [score] | [score] | [tag] |

## 8.5 Comparing decoding methods

Compare Viterbi with a most-frequent-tag baseline and exhaustive enumeration of all possible tag sequences. The baseline chooses the most common tag for each word without considering the complete sequence. Exhaustive enumeration checks every possible tag sequence and is expensive. Viterbi uses dynamic programming to avoid repeating calculations.

| **Method** | **Accuracy** | **Operations for selected sentence** | **Comment** |
|---|---|---|---|
| Most-frequent-tag baseline | [value]% | [count] | Simple baseline |
| Exhaustive sequence scoring | [value]% | [count] | Very expensive as tags increase |
| Viterbi | [value]% | [count] | Efficient dynamic programming |

## 8.6 Unknown words

An HMM may not have an emission probability for an unseen word. A practical method uses suffixes, capitalisation, digit patterns or an open-class prior. For example, an unseen capitalised token after “to” or “from” may be considered a place name, but this is only a clue. A suffix-based model can learn that certain endings are common for nouns, adjectives or adverbs.

Evaluate unknown-word accuracy separately by selecting test tokens absent from the training vocabulary. Report the number of OOV tokens and the percentage tagged correctly.

# TASK 9 – Transformation-Based (Brill) Tagging 4 Marks

**Guiding Enquiry**

Can a tagger learn its own correction rules, and are the rules it learns worth reading?

**Investigate and interpret:**

- Describe transformation-based learning: the initial-state annotator, the templates, the greedy selection of the transformation that corrects the most errors, and the stopping condition.
- Define at least six rule templates over the preceding tag, the following tag, the surrounding words and the word's own form, and justify the window you chose.
- Run the learner from a most-frequent-tag initial state and report the first ten transformations learnt in order, with the number of errors each one corrected and introduced.
- Explain why the learnt rules are readable in a way that HMM probabilities are not, and show one learnt rule that encodes a genuine linguistic generalisation and one that is an artefact of your training data.
- Measure accuracy against the number of transformations applied, plot the curve, and identify the point beyond which further rules overfit.

**Evidence Expected**
A working Brill learner with its learnt rule list, and an accuracy curve identifying the overfitting point.

# ANSWER

## 9.1 Basic idea

Transformation-based learning begins with an initial annotation, often the most-frequent-tag assignment for each word. It then learns correction rules from errors. At each iteration, it searches candidate transformations and selects the rule that corrects the largest number of errors, while accounting for errors it may introduce. Learning stops when no rule exceeds the chosen score threshold or a maximum number of rules is reached.

## 9.2 Rule templates

Templates define the forms of transformations the learner is allowed to consider. Examples include:

Change the current tag from A to B if the previous tag is C.

Change the current tag from A to B if the next tag is C.

Change the current tag from A to B if the previous word is W.

Change the current tag from A to B if the next word is W.

Change the current tag from A to B if the word ends with suffix S.

Change the current tag from A to B if the word begins with prefix P.

Change the current tag from A to B if the previous two tags are C and D.

Change the current tag from A to B if the next word is a known station name.

Use at least six templates. A small context window reduces computation and avoids rules that depend on distant words. A larger window may capture more context but can overfit the training corpus.

## 9.3 Learned transformations

Run the learner from a most-frequent-tag initial state. Record the first ten transformations in the exact order learned. For each rule, report how many training errors it corrected and how many errors it introduced. Do not invent the rule list; it depends on the annotated corpus and template set.

| Order | Transformation rule | Errors corrected | Errors introduced |
|---|---|---|---|
| 1 | [learned rule] | [value] | [value] |
| 2 | [learned rule] | [value] | [value] |
| 3 | [learned rule] | [value] | [value] |
| 4 | [learned rule] | [value] | [value] |
| 5 | [learned rule] | [value] | [value] |
| 6 | [learned rule] | [value] | [value] |
| 7 | [learned rule] | [value] | [value] |
| 8 | [learned rule] | [value] | [value] |
| 9 | [learned rule] | [value] | [value] |
| 10 | [learned rule] | [value] | [value] |

## 9.4 Readability and overfitting

Brill rules are readable because they express explicit corrections such as changing a tag when the preceding tag has a certain value. HMMs represent behaviour through many transition and emission probabilities, which are less directly interpretable. A learned rule may express a genuine grammatical pattern, or it may memorise a peculiarity of the training data.

Measure tagging accuracy after applying 0, 1, 2, … transformations. Plot accuracy against the number of rules. Training accuracy may continue to increase while development accuracy stops improving or decreases. This point indicates overfitting; stop before adding rules that do not generalise.

| Number of rules | Training accuracy | Development accuracy |
|---|---|---|
| 0 | [value] | [value] |
| 1 | [value] | [value] |
| 5 | [value] | [value] |
| 10 | [value] | [value] |
| [best point] | [value] | [value] |
| [later point] | [value] | [value] |

# TASK 10 – Implement and Test the Models 3 Marks

**Guiding Enquiry**

Can a working implementation reproduce the behaviour you derived by hand?

**Investigate and interpret:**

- Implement in Python the counting and vocabulary pipeline, the unigram, bigram and trigram models with every smoothing method, backoff and interpolation, and all three taggers, using the standard library and NLTK only.
- Run every model over the same splits and report, for each: training time, model size in memory, held-out perplexity or tagging accuracy, and tagging speed in tokens per second.
- Include test cases for the zero-probability utterance from Task 2, the λ weights from Task 4, the trellis from Task 8 and the first transformations from Task 9, checking each against your hand-computed result.
- Present the results in comparable tables and at least two plots — for example perplexity against n-gram order, and tagging accuracy against training-set size.

**Evidence Expected**

Working, documented code, a reproducible test procedure, and results tables and graphs that match the hand calculations.

# ANSWER

## 10.1 Implementation plan

Implement the pipeline in Python using the standard library and NLTK only, as required by the assignment. Keep preprocessing, model estimation, decoding and evaluation in separate functions so that each part can be tested independently.

| **Module** | **Main responsibility** |
|---|---|
| preprocess.py | Read utterances, normalise and tokenize |
| vocabulary.py | Count tokens, build vocabulary and map <UNK> |
| ngram.py | Unigram, bigram and trigram counts and probabilities |
| smoothing.py | Add-one, add-k and Good–Turing methods |
| backoff.py | Katz backoff and interpolation |

| | |
|---|---|
| metrics.py | Entropy, cross-entropy, perplexity and WER |
| tagset.py | Tag definitions and PTB mapping |
| rule_tagger.py | Dictionary and contextual rules |
| hmm_tagger.py | Transition/emission estimates and Viterbi |
| brill_tagger.py | Templates, learning and transformation sequence |
| evaluate.py | Shared evaluation and result tables |

### 10.2 Example core Python structure

The following short example illustrates how to count unigrams and bigrams. Extend it for trigrams, smoothing and the taggers. Use your own corpus files and preserve the same train/development/test split across all experiments.

```
from collections import Counter

sentences = [
   ["book", "two", "tickets"],
   ["bus", "timing", "please"]
]

unigrams = Counter()
bigrams = Counter()

for sent in sentences:
   words = ["<s>"] + sent + ["</s>"]
   unigrams.update(sent)
   bigrams.update(zip(words, words[1:]))

print("Unigrams:", unigrams)
print("Bigrams:", bigrams)
```

This is a demonstration only, not a complete implementation of the assignment. The final code

must load the documented corpus, use the prescribed splits and implement every method evaluated in Tasks 2–9.

### 10.3 Reproducible tests

Set and record a fixed random seed for any random split, sampling or generation. Add test cases for: the zero-probability utterance from Task 2; the deleted-interpolation weights from Task 4; the Viterbi trellis from Task 8; and the first transformations from Task 9. Compare program output with the hand calculations. A mismatch should be investigated before reporting the results.

### 10.4 Performance comparison

For every model, record training time, memory size, held-out perplexity or tagging accuracy, and tagging speed in tokens per second. Use the same machine and evaluation data for fair comparison. Include at least two plots, such as perplexity against n-gram order and tagging accuracy against training-set size.

| **Model** | **Training time** | **Memory** | **Perplexity / accuracy** | **Speed (tokens/s)** |
|---|---|---|---|---|
| Unigram / bigram / trigram | [value] | [value] | [value] | [value] |
| Smoothed / backoff model | [value] | [value] | [value] | [value] |
| Rule-based tagger | [value] | [value] | [value] | [value] |
| HMM tagger | [value] | [value] | [value] | [value] |
| Brill tagger | [value] | [value] | [value] | [value] |

**TASK 11 – Compare, Recommend and Defend the Design 2 Marks**

**Guiding Enquiry**

Which language model and which tagger would you deploy on the helpline, and what evidence supports the choice?

**Investigate and interpret:**

- Compare the smoothing and backoff strategies on perplexity, word error rate after rescoring, memory and speed, and compare the three taggers on accuracy, accuracy on ambiguous and unknown tokens, speed and ease of maintenance.
- Recommend one language model and one tagger, state the expected accuracy, the failure modes, and what a caller would actually experience when the system fails.
- Discuss the manual effort involved in annotation and rule writing, and identify the point at which more annotated data would help more than more rules.

**Evidence Expected**

A justified recommendation supported by theory and measurements; choosing the highest accuracy alone is not sufficient.

# ANSWER

### 11.1 Language-model comparison

Compare smoothing and backoff strategies using more than one criterion. Perplexity measures prediction quality, while WER measures whether rescoring improves the actual transcript. Memory and speed matter because the helpline processes thousands of calls per day. A model with slightly better perplexity may not be the best deployment choice if it is much slower or uses excessive memory.

### 11.2 Tagger comparison

Compare the rule-based, HMM and Brill taggers using overall accuracy, ambiguous-token accuracy, unknown-token accuracy, speed and maintainability. A rule-based tagger is transparent but requires manual rule writing. An HMM learns probabilities from annotation data and uses efficient Viterbi decoding, but unknown words and independence assumptions are limitations. A Brill tagger produces readable correction rules but may overfit if too many transformations are learned.

| **Criterion** | **Rule-based** | **HMM** | **Brill** |
|---|---|---|---|
| Overall accuracy | [value] | [value] | [value] |

| | | | |
|---|---|---|---|
| Ambiguous-token accuracy | [value] | [value] | [value] |
| Unknown-token accuracy | [value] | [value] | [value] |
| Speed | [value] | [value] | [value] |
| Maintenance effort | [low/medium/high] | [low/medium/high] | [low/medium/high] |

### 11.3 Recommendation

Select one language model and one POS tagger only after completing the measurements. The recommendation should balance accuracy, robustness to unseen words, speed, memory and maintenance. State the expected accuracy using the measured held-out result, describe common failure modes and explain what the caller may experience. For example, a wrong destination name may cause an incorrect route result, while a wrong tag may cause the information-extraction component to miss a date or fare.

Manual effort must include data collection, transcription checking, tagset design, annotation, adjudication, rule writing, testing and maintenance. Estimate hours and explain whether additional annotated data would improve performance more than writing additional rules.

| **Effort item** | **Estimated hours** | **How estimated** |
|---|---|---|
| Corpus preparation | [hours] | [method] |
| Tagset design | [hours] | [method] |
| Annotation and review | [hours] | [method] |
| Rule writing / correction | [hours] | [method] |
| Implementation and testing | [hours] | [method] |
| Ongoing maintenance | [hours/month] | [method] |

**DAY-4 DESIGN CHALLENGE**

**TASK 12 – Re-evaluate and Adapt the Design 8 Marks**

**Changed Requirement**
The helpline now also answers metro and intercity rail enquiries, so hundreds of new station names, train names and fare categories enter the vocabulary every month. Callers increasingly speak code-mixed sentences, for example "Chennai to Madurai bus timing enna", and the recogniser transcribes those words phonetically and inconsistently. More than fifteen per cent of the word tokens in new calls were never seen in training, many of them exactly the proper nouns that carry the caller's intent.

Re-evaluate your original design under the changed conditions. You may:

**RETAIN → MODIFY → or REDESIGN**

your original solution.

**A. Impact Analysis – 2 Marks**

**Guiding Enquiry**

What exactly has changed, and which of your original assumptions no longer hold?

- Identify what the new input breaks: a closed vocabulary, a stable unknown-word rate, English-only word classes, transition probabilities estimated on a narrower domain, and a tagset that has no category for code-mixed tokens.
- Explain why the out-of-vocabulary words are the worst possible ones to lose, given that they are mostly the place names the enquiry turns on.
- Estimate the effect on perplexity and on tagging accuracy, reasoning from the OOV rate you measured in Task 1 and the unknown-word accuracy you measured in Task 8.

**Evidence Expected**

A clear analysis of how the new input exposes gaps in the original design.

# ANSWER

## 12A. Impact analysis

A high OOV rate affects both the language model and the tagger. If a new station name is mapped to <UNK>, the model loses information that may be essential for identifying the destination. The tagger may also assign a generic unknown tag instead of PROPN or a more specific place-name category. Code-mixed words may not match the English word-class patterns or the original transition probabilities.

Use the original OOV rate from Task 1 and unknown-token tagging accuracy from Task 8 to estimate the expected degradation. Keep the estimate separate from measured results. The new scenario states that more than 15% of tokens are unseen; report the exact value measured on

the new sample.

**B. Evaluation of the Original Design – 2 Marks**

**Guiding Enquiry**

How would your recommended model and tagger behave on the new calls?

- Run or hand-trace at least three new utterances — one with an unseen station name, one code-mixed, one with both — through your language model and your chosen tagger, and record what fails first and why.
- Measure the degradation: held-out perplexity, word error rate after rescoring, and tagging accuracy on the new sample against your original baselines.
- Identify which component degrades fastest, and whether a different approach — a class-based or character-level model, an open-vocabulary tagger, a gazetteer of station names — would handle the new input more economically.

**Evidence Expected**

A technically supported evaluation of the original design with traces or measurements.

# ANSWER

**12B. Evaluate the original design**

Test at least three new utterances: one containing an unseen station name, one code-mixed utterance and one containing both. Trace each through the original language model and selected tagger. Record the first failure: vocabulary mapping, probability scoring, POS assignment or downstream extraction.

| **New utterance type** | **Example** | **Likely point to inspect** | **Measured result** |
|---|---|---|---|
| Unseen station name | “bus to [new station]” | OOV mapping and proper-noun tag | [result] |
| Code-mixed | “timing enna” | Language identification and tag coverage | [result] |
| Both | “[new station] to city timing enna” | OOV plus mixed-language context | [result] |

Measure new-sample perplexity, WER after rescoring and tagging accuracy. Compare them with the original baseline using the same evaluation procedure. Candidate approaches include a station-name gazetteer, class-based n-grams, character-level language models and an open-vocabulary tagging strategy.

**C. Adaptation or Redesign – 2 Marks**

**Guiding Enquiry**

Should the models be retrained, extended or replaced?

- Choose retain, modify or redesign — for example adding a station-name gazetteer and a class-based n-gram over it, re-estimating λ weights on new data, extending the tagset with a code-mixed category, or adding a character n-gram model for unseen words.
- Describe the revised design and state exactly what changes in the vocabulary, the smoothing, the backoff weights, the tagset and the tagger.
- Determine the effect on perplexity, tagging accuracy, retraining frequency and annotation effort, and say how often the model must now be refreshed.

**Evidence Expected**

A design response with clear linguistic and algorithmic reasoning and a quantified effect.

# ANSWER

**12C. Adaptation or redesign**

A practical decision is to modify the original system rather than discard every component. Keep the useful n-gram and tagging framework, but add domain resources and retrain using representative new calls. Possible changes include:

Vocabulary: update station, train, route and fare names regularly; preserve important proper nouns rather than mapping all rare names to one <UNK> token.

Gazetteer: maintain a list of known stations, train names and transport locations, including spelling variants.

Language model: add class-based representations for station names and use character-level or subword evidence for unseen forms.

Smoothing and backoff: re-estimate discounts and interpolation weights using new development data that includes rail and code-mixed calls.

Tagset: add a code-mixed or language-specific category only when annotation evidence shows that it is needed; preserve PROPN for place names.

Tagger: use suffix, character and gazetteer features for unknown words; retrain transition and emission probabilities on the expanded domain.

Refresh schedule: monitor new-name frequency and OOV rate, then update the vocabulary and models on a regular schedule. State the chosen schedule and justify it from measured drift.

**D. Evidence-Based Defence – 2 Marks**

**Guiding Enquiry**

Why is your final decision the right one, given the evidence you generated?

- Use your perplexity tables, rescoring results, tagging accuracies, error analyses and effort estimates.
- Show that the decision follows logically from that evidence, and state its remaining limitations — including what an n-gram model can never capture, however much data it is given.

**Evidence Expected**

A well-supported final decision integrating theory, experiments and the changed requirements.

# ANSWER

**12D. Evidence-based defence**

Defend the final design using the perplexity tables, rescoring WER, tagging accuracy, unknown-word results, error analysis and effort estimates. A decision should follow from the evidence rather than from the assumption that the most complex model is best.

An n-gram model has a fundamental limitation: it uses only a fixed number of preceding words. Even with more data, it cannot fully represent long-distance dependencies, deep meaning, speaker intention or all context-dependent interpretations. A gazetteer and character model can reduce unknown-name errors, but they do not solve every language-understanding problem.

| **Decision area** | **Final choice** | **Evidence supporting the choice** |
|---|---|---|
| Overall strategy | Retain / Modify / Redesign: [choose] | [measurements] |
| Language model | [selected model] | [perplexity and WER] |
| POS tagger | [selected tagger] | [accuracy and robustness] |

| | | |
|---|---|---|
| Unknown names | [gazetteer / character model / other] | [OOV evaluation] |
| Retraining schedule | [schedule] | [vocabulary drift] |
| Remaining limitation | [limitation] | [error analysis] |

**MARK DISTRIBUTION**

| Task | Marks |
|---|---|
| Task 1 – Counting Words and Building the Vocabulary | 3 |
| Task 2 – Unsmoothed N-gram Models | 4 |
| Task 3 – Smoothing and Discounting | 5 |
| Task 4 – Backoff and Deleted Interpolation | 4 |
| Task 5 – Entropy, Cross-Entropy and Perplexity | 4 |
| Task 6 – English Word Classes and Tagset Design | 4 |
| Task 7 – Rule-Based Part-of-Speech Tagging | 4 |
| Task 8 – Stochastic (HMM) Tagging and the Viterbi Algorithm | 5 |
| Task 9 – Transformation-Based (Brill) Tagging | 4 |
| Task 10 – Implement and Test the Models | 3 |
| Task 11 – Compare, Recommend and Defend the Design | 2 |
| **Subtotal** | **42** |
| Task 12 – Day-4 Design Challenge | 8 |
| **TOTAL** | **50** |